Indian Ad Classification: Full Project Guide

Oct 30, 2026 · @snehal

Where we are now

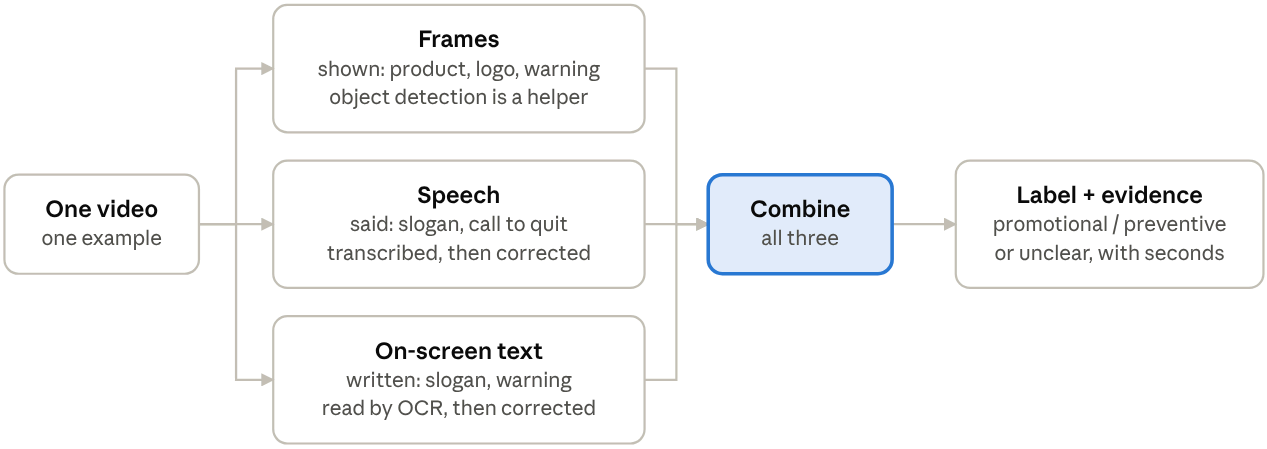
The collection register holds 23 ads (10 promotional, 13 preventive), all still `link_only`. That is a good pilot start, but five problems need fixing before we move on.

1. **Product shortcut (biggest problem).** Every promotional ad is pan masala or elaichi, and every preventive ad is tobacco. A model would learn "pan masala = promotional, tobacco = preventive" instead of learning the message. We need preventive ads about alcohol or pan masala/gutka (drink-driving, anti-alcohol PSAs) and promotional ads for alcohol or tobacco-linked brands (surrogate ads). There are no alcohol ads yet.
2. **Source shortcut.** All promotional ads come from brands and all preventive ones from NGOs or government, so the model can learn the production style. Add a few outliers, such as a brand-made responsible-use ad or an NGO ad that shows a product.
3. **Duplicate link.** B_0002 and B_0003 share one video ID (`0kHeW6_X6K4`) but have different notes. One of them is wrong.
4. **Missing `campaign_id`.** Several ads are versions of one campaign: "When You Quit" (B_0003, B_0013), "I Don't Believe" (B_0008, B_0011), and the Sunita and Mukesh testimonials. Give each campaign one ID so its versions stay in the same train/test split.
5. **Small clean-ups.** Fill `youtube_video_id` from the URL, remove the leading space in `cigarette_smoking`, and fill `promotion_type` and `source_type` for every row.

The big idea

One video is one example, and we never classify single frames. The model reads three streams from each video and decides the overall message. We are not only detecting whether an object is present: we are deciding whether the ad promotes or prevents, so we need text and speech as well as pictures.

Three streams from one video combine into one label



pipeline · 3 streams into 1 label

A cigarette appears in both promotional and preventive ads, so speech, text and tone decide the label, not the object alone.

Steps, in order

The project runs in nine steps, from cleaning the register to a working demo.

1. **Fix the register.** Do the clean-ups above and collect more ads so every product has both classes. Aim for about 100 to 200 ads in total.
2. **Download the videos privately.** Use yt-dlp (check YouTube's terms first), save each as `videos/AD_0001.mp4`, and set `video_status = downloaded`. Share only links and labels later, not the videos.
3. **Write the labeling rules and label.** Use the guide's label definitions. Two people label each ad independently, a third resolves disagreements, and we compute Cohen's kappa on the first 20 to 30 ads. Fill `final_label` and `label_reason`.
4. **Extract the three streams.**
   - **Audio:** pull a 16 kHz `.wav` from each video with ffmpeg.
   - **Frames:** save 1 frame per second, plus extra frames at scene changes so brief logos and warnings are not missed. This gives `frames/` and `frames.csv`.
   - **Speech:** run Whisper (it handles Hindi, Marathi and English) to get timed segments, then correct them by listening. This gives `speech.csv`.
   - **On-screen text:** run EasyOCR or PaddleOCR (Hindi plus English) on the frames, merge repeated text, and correct it by hand. This gives `screen_text.csv`.

5. **Turn each stream into numbers (features).**
    - **Frames:** run each frame through a pretrained image model (CLIP or SigLIP) and average the results into one vector per video.
    - **Speech and screen text:** embed them with a multilingual text model (for example multilingual-e5 or MuRIL).
    - **Optional helpers:** flags for "health warning text present" and "product or smoking visible", from OCR keywords or a small detector.
    - With only 100 to 200 videos, use these frozen pretrained features. Do not train a deep network from scratch.
6. **Train baselines, simplest first.**
    - Text-only (speech plus OCR) with logistic regression.
    - Frames-only with logistic regression on the CLIP vectors.
    - Combined: join all vectors and train one classifier.
    - Add a zero-shot baseline too: give a vision-language LLM the frames and transcript and ask for the label. Comparing these baselines is one of the main results.
7. **Evaluate properly.**
    - Split by campaign (70/15/15), never by frame.
    - Report macro F1, plus precision and recall per class.
    - Run the shortcut tests: promotional ads that carry a warning, and preventive ads that show a brand or product.
    - Break results down by product, language and source type.
8. **Explain the decision.** For each video, output the label plus the seconds and modality that drove it, for example "0:06 speech praises the brand, so promotional." Compare these against the human evidence, which is why `evidence.csv` is built for a smaller subset of ads.
9. **Demo.** The final pipeline takes a video, runs extraction and then the model, and shows the label with a timeline of evidence.

Which models we use

We do not pick one model. The pipeline uses a small set of pretrained models, one per job, and a simple classifier on top.

Job	Model	Why
Speech to text	Whisper (medium or large-v3)	Handles Hindi, Marathi and English, with timestamps
Text on screen	EasyOCR or PaddleOCR	Reads Hindi and English on frames
Frames to numbers	CLIP or SigLIP, kept frozen	Captures what is shown (product, party scene, warning box) without training
Text to numbers	multilingual-e5 or MuRIL	Works on mixed Hindi-English transcripts and OCR text
Final decision	Logistic regression, then a small MLP if needed	Works with 100 to 200 videos
Optional detector	YOLO (cigarette, bottle, pack, warning box)	Only a helper and a source of evidence, not the core
Zero-shot baseline	A vision-language LLM given frames plus transcript	A comparison point for the trained model

Each stream becomes a vector (frames averaged into one, speech and OCR text embedded separately). The three vectors are joined, and the classifier predicts promotional, preventive or unclear.

What gets trained and what does not

Only the final classifier is trained. It takes the combined vector for each video and predicts promotional, preventive or unclear, learning from your labeled ads: the inputs are the extracted features, and the target is your `final_label`.

Whisper, OCR, CLIP and the text embedding model are used as they are. They only turn each video into numbers.

The order is:

1. Label the ads.
2. Extract frames, speech and on-screen text.
3. Convert each stream to a vector with the pretrained models.
4. Join the vectors for each video.

5. Split by campaign into train, validation and test sets.
6. Train the classifier on the train set, tune it on validation, and report results once on test.

Each ad is one row, with one input vector and one label. Training takes minutes on a laptop, not hours on a GPU, because the model is small.

If the results are weak, the next step would be fine-tuning part of a pretrained model (for example the image encoder). That needs more labeled data, so it is an extension, not the plan.

The final model: name, features and training

The model is a multimodal late-fusion classifier built from three small logistic regression models. Its name: "CLIP + multilingual-e5 + Logistic Regression, late fusion." If it underperforms, the next step is a small MLP on the joined vectors.

Input features (per video):

Stream	How it becomes numbers	Size
Frames	CLIP image embedding of each frame, averaged (mean-pooled) over the video	512
Speech	Transcript embedded with multilingual-e5	768
On-screen text	OCR text embedded with the same model	768
Helper flags (optional)	Warning text present, smoking or bottle visible (from OCR keywords or CLIP prompts like "a person smoking")	a few numbers

What stays out of the model: `source_type`, `platform`, `final_label`, `label_reason`, and the annotator columns. Also keep `product_category` and `brand` out of the input, because they cause the product shortcut. Use them only afterwards to check results by group.

How it trains:

1. Train three separate logistic regression models: one on frame vectors, one on speech vectors, one on OCR vectors. Each predicts promotional vs preventive.
2. Combine their probabilities (average them, or use a tiny logistic regression on top).

3. Use strong regularization and balanced class weights. With about 2,000 input numbers and only 100 to 200 videos, a flexible model would just memorize.
4. Run cross-validation grouped by `campaign_id` (scikit-learn `GroupKFold`) to tune settings. Then score once on the held-out test set, using macro F1 plus per-class precision and recall.

Why late fusion: each stream gives its own probability, so we can say "speech said promotional (0.9), frames said neutral (0.5)." That feeds directly into the explanation step and shows which modality drove the decision.

The comparison we report is text-only, frames-only and combined, plus the zero-shot LLM baseline.

The pipeline in one paragraph

For each video, we first pull out three things: the frames (pictures), the speech (Whisper converts the audio to text), and the on-screen text (OCR reads words like slogans and health warnings). Each of these is then turned into a list of numbers by a ready-made pretrained model: CLIP for the pictures, and a multilingual text model for the speech and screen text, so every video becomes a fixed set of numbers describing what is shown, said and written. We don't train those ready-made models. We only train one small, simple classifier (logistic regression) on top: we show it the numbers from our labeled videos along with your label (promotional or preventive), and it learns which patterns of shown, said and written content go with each label. We train one small classifier per stream, then average their answers into the final decision, which also tells us which stream (frames, speech or text) pushed the result. Finally, we test it on videos from campaigns it never saw during training, to check that it learned the message and didn't just memorize ads.

A small worked example

The ads and numbers below are invented, only to show how the pipeline works.

Two training videos, already labeled by you

- **Ad 1** (promotional): friends celebrate with a pan masala pack. A voice says "Celebrate every moment with Brand X." A tiny health warning shows at the end.
- **Ad 2** (preventive): a patient talks about mouth cancer. A voice says "Tobacco destroyed my life. Quit today." The screen says "Tobacco kills."

Step 1: Extract the three streams

Stream	Ad 1	Ad 2
Frames	party, smiling people, pack	hospital bed, sad face, doctor
Speech (Whisper)	"Celebrate every moment with Brand X"	"Tobacco destroyed my life. Quit today."
Screen text (OCR)	"Brand X, make it special", "health warning"	"Tobacco kills"

Step 2: Turn each stream into numbers

The pretrained models do this, and we don't train them. Each stream becomes a list of numbers that captures its meaning. Similar meanings get similar numbers:

- Ad 1 speech, "celebrate... Brand X": roughly `[0.8, 0.1, ...]`, a cheerful, brand-praising direction.
- Ad 2 speech, "destroyed my life... quit": roughly `[0.1, 0.9, ...]`, a harm and quitting direction.

Frames work the same way: party pictures get numbers close to other party pictures, and hospital pictures get numbers close to other hospital pictures.

Step 3: Train the small classifiers

We train three small logistic regression classifiers, one per stream. Each sees these numbers plus your labels and learns a rule like:

- Speech classifier: "cheerful, brand-praising numbers go with promotional; harm and quitting numbers go with preventive."
- Frames classifier: "party-like pictures go with promotional; hospital-like pictures go with preventive."
- Text classifier: "slogans go with promotional; 'tobacco kills' goes with preventive."

With 100 to 200 real videos, these rules become much more reliable than with two.

Step 4: Predict on a new video (Ad 3)

Ad 3 is a new video from a campaign the model has never seen. It shows a cricketer smiling with a pack, says "Play bold, taste bold," and has a tiny warning at the end.

Stream	Prediction
Frames	promotional, 0.85
Speech	promotional, 0.90
Screen text	promotional, 0.60 (the warning pulls it down)

The average of the three is about 0.78, so the result is **promotional**, with the speech stream pushing hardest. The small warning does not flip the result, which is the behavior the labeling rules require.

Step 5: Check it fairly

We test on campaigns the model never trained on. If it gets those right, it learned the message. If it fails on them, it had memorized specific ads.

What to do this week

Finish these four tasks before touching any model.

- [ ] Clean the register and fix the duplicate link (B_0002 and B_0003).
- [ ] Find alcohol ads in both classes, plus preventive pan masala or alcohol ads.
- [ ] Download the videos.
- [ ] Write the one-page labeling rules and have two people label the first 20 to 30 ads independently.